

Inference-Time Contextual State Assembly

Separating Evolving Operational State from Foundation Model Parameters

Author:

Andrew Fereday Glenn (in collaboration with Mia, ChatGPT 5.x)
Systems Architect & Independent Researcher in AI Continuity and Memory
2023-2026

LinkedIn: <https://www.linkedin.com/in/andyglenn/>

Version: 1.0

First Published: Friday, 7 August 2026

Last Updated: Friday, 7 August 2026

Status: Final

Abstract

Foundation models have demonstrated remarkable capabilities across a wide range of artificial intelligence applications. However, many persistent AI systems require operational state—including identity, memory, environmental awareness, user context, accumulated knowledge, and evolving application data—to change continuously throughout their lifetime. Traditional approaches typically address changing behaviour through model adaptation techniques such as fine-tuning or Low-Rank Adaptation (LoRA), or by augmenting inference with Retrieval-Augmented Generation (RAG). While highly effective within their intended domains, these approaches do not explicitly address the architectural separation between relatively stable reasoning capability and rapidly evolving operational state.

This paper introduces **Inference-Time Contextual State Assembly (ICSA)**, an architectural pattern in which application-specific contextual state remains external to the foundation model and is dynamically assembled immediately prior to inference. Rather than continually encoding evolving knowledge and state into model parameters, ICSA reconstructs an operational representation of the current world by combining information from multiple contextual sources, allowing the foundation model to reason over an accurate and continuously evolving representation of the present context.

The proposed architecture is illustrated through **Brain v2**, a cognitive architecture developed to support persistent digital personas. Within Brain v2, contextual state is assembled from multiple heterogeneous sources including long-term knowledge repositories, structured memory, historical interaction, identity, environmental observations, and application-specific information. Although persistent digital personas provide a

demanding case study, the underlying architectural principles generalise to a wide range of long-lived AI systems, including corporate knowledge assistants, engineering support systems, scientific research platforms, and other applications requiring continuously evolving contextual state.

The paper argues that separating reasoning capability from operational state provides a scalable and maintainable architectural pattern for persistent AI systems. Rather than replacing existing techniques such as fine-tuning or Retrieval-Augmented Generation, Inference-Time Contextual State Assembly complements them by identifying a distinct architectural layer responsible for representing rapidly evolving contextual state. This separation allows application state to evolve continuously while remaining compatible with improvements in underlying foundation models, reducing dependence on continual model adaptation and providing a flexible framework for future cognitive architectures.

1. Introduction

The architectural principles described in this paper evolved incrementally through several earlier experimental systems. Initial work with stateless language models demonstrated that maintaining coherent behaviour within a persistent fictional environment required comprehensive reconstruction of operational context prior to inference. Although primitive in implementation, these early experiments established the architectural direction that later evolved into Inference-Time Contextual State Assembly.

Large Language Models (LLMs) have demonstrated remarkable capabilities across a broad range of domains, from natural language understanding and software development to scientific reasoning and creative writing. As the capabilities of foundation models continue to improve, increasing attention has shifted towards the construction of higher-level cognitive architectures capable of supporting persistent agents, domain-specific assistants, and long-lived digital personas.

Current approaches to adapting foundation models generally fall into two broad categories. The first modifies the model itself through techniques such as fine-tuning, continual pre-training, or parameter-efficient adaptation methods including Low-Rank Adaptation (LoRA). The second leaves the underlying model unchanged while augmenting inference through external knowledge sources, most commonly Retrieval-Augmented Generation (RAG).

Both approaches have proven highly effective within their intended domains. Fine-tuning enables models to acquire specialised behaviours or domain expertise, while traditional RAG systems provide access to external documents and knowledge repositories without modifying model parameters. These techniques are complementary rather than mutually exclusive, and each addresses a distinct class of problems.

This paper examines a different architectural problem: the construction of persistent, continuously evolving cognitive systems in which knowledge, identity, relationships, environmental awareness, and lived experience change far more rapidly than the underlying reasoning engine itself.

Rather than continually adapting model parameters to reflect this evolving state, we explore an alternative architecture in which the foundation model remains largely unchanged while a rich contextual state is dynamically reconstructed prior to every inference. This reconstructed state extends beyond traditional document retrieval to include identity, memory, environmental context, recent experience, external observations, and other application-specific information relevant to the current moment.

Although the ideas presented are illustrated through the Brain v2 cognitive architecture and its persistent digital personas, the underlying principles are considerably more general. We argue that Inference-Time Contextual State Assembly provides a scalable architectural pattern applicable to any system in which operational state evolves continuously while the underlying reasoning capabilities remain relatively stable.

2. The Current Paradigm

2.1 Model Adaptation

Since the emergence of modern Large Language Models (LLMs), considerable effort has been directed towards adapting foundation models for specialist applications. While foundation models provide broad reasoning capabilities, many practical deployments require domain-specific knowledge, behavioural modification, or specialised expertise beyond that available in the base model.

A common approach is to modify the model itself through additional training. This may take the form of full fine-tuning, continued pre-training, or parameter-efficient adaptation techniques such as Low-Rank Adaptation (LoRA). These methods enable developers to specialise models for particular tasks while retaining much of the capability learned during large-scale pre-training.

Model adaptation has demonstrated considerable success across a wide range of domains including software engineering, legal analysis, medical applications, scientific research, and domain-specific conversational systems. By embedding additional knowledge or behavioural characteristics directly into model parameters, specialised models can often outperform general-purpose models within narrowly defined tasks.

However, modifying model parameters necessarily produces a static representation of knowledge at the point of training. As underlying information evolves, additional training or adaptation may be required to maintain alignment with the current state of the domain.

For applications in which knowledge changes relatively slowly, this approach can be highly effective. For systems whose operational state evolves continuously, however, repeated model adaptation introduces increasing computational cost, maintenance overhead, and deployment complexity.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) offers an alternative approach by leaving the underlying model unchanged while supplying additional information during inference. Rather than embedding knowledge directly into model parameters, external information is retrieved dynamically and incorporated into the model's prompt.

In its most common form, RAG is used to enable interaction with external knowledge repositories such as technical documentation, research papers, corporate documentation, or other structured collections of text. Retrieved documents provide additional factual grounding while allowing the underlying foundation model to remain unchanged.

This approach offers several practical advantages. Knowledge repositories can be updated without retraining the model, multiple independent knowledge bases can coexist, and information can be corrected or expanded without modifying model parameters. These

characteristics have made RAG one of the dominant architectural patterns for enterprise AI systems.

Traditional RAG systems, however, typically focus on document retrieval. Their primary objective is to identify relevant information in response to a user's query rather than reconstructing a continuously evolving contextual state.

2.3 Limitations for Continuously Evolving Systems

Both model adaptation and Retrieval-Augmented Generation address important challenges within modern AI systems. However, both are generally designed around applications in which either model behaviour or external knowledge is the primary concern.

Persistent cognitive architectures introduce a different class of problem.

In continuously evolving systems, operational state extends beyond factual knowledge. Identity, recent experiences, interpersonal relationships, environmental awareness, current objectives, historical interactions, and accumulated learning all contribute to the context within which reasoning takes place. Much of this information changes continually, often from one interaction to the next.

Encoding such rapidly evolving state directly into model parameters is neither practical nor computationally efficient. Equally, treating these diverse contextual elements as isolated documents within a conventional retrieval system fails to capture the interconnected nature of persistent cognitive state.

This distinction motivates the architectural approach explored in the remainder of this paper. Rather than viewing context as a collection of documents to retrieve, we instead consider context as a dynamic state to be reconstructed prior to each inference.

3. Inference-Time Contextual State Assembly

3.1 Architectural Philosophy

The central premise of Inference-Time Contextual State Assembly (ICSA) is that rapidly evolving operational state should remain external to the foundation model and be reconstructed dynamically at inference time rather than encoded into model parameters.

This approach begins by separating two fundamentally different responsibilities within the cognitive architecture.

The first is the **reasoning engine**. The foundation model provides the general capabilities required for language understanding, logical reasoning, synthesis, abstraction, planning, and natural language generation. These capabilities typically evolve relatively slowly as improved foundation models become available.

The second is **contextual state**. Unlike the reasoning engine, contextual state evolves continuously throughout the lifetime of the system. It encompasses not only factual knowledge but also identity, recent experience, accumulated learning, environmental awareness, relationships, goals, and other information specific to the current moment.

Rather than embedding this continually evolving state within model parameters, Inference-Time Contextual State Assembly assembles an appropriate working representation immediately prior to each inference. The foundation model therefore reasons over the reconstructed contextual state rather than relying solely upon information encoded during pre-training or subsequent fine-tuning.

This separation allows the reasoning engine and contextual state to evolve independently. Improvements to the underlying foundation model do not require reconstruction of contextual state, while changes to contextual state occur continuously without requiring additional model training.

3.2 Contextual State

Within Inference-Time Contextual State Assembly, contextual state extends significantly beyond the traditional concept of document retrieval.

Instead of viewing external knowledge as a collection of independent documents, ICSA considers all relevant information required to establish the current cognitive state of the system.

Depending upon the application, contextual state may include:

- Persistent identity or persona definition
- Long-term knowledge repositories
- Episodic memories and historical interactions
- Relationships between entities
- User preferences and behavioural patterns
- Environmental observations
- Current goals and active tasks
- Recently acquired information
- Structured metadata describing prior experiences
- External information sources relevant to the current interaction

Not every application requires every component. Rather, contextual state is assembled from those information sources most relevant to the current inference.

Importantly, these information sources remain independently maintainable. New observations, experiences, or documents can be incorporated continuously without modifying the underlying reasoning engine.

3.3 Contextual Reconstruction

Immediately prior to inference, relevant elements of contextual state are selected and assembled into a coherent working representation.

Unlike conventional prompt construction, this process is not simply the concatenation of retrieved documents. It is the reconstruction of the current operational state required by the reasoning engine.

Selection may be guided by semantic similarity, temporal relevance, relational structure, environmental context, application-specific heuristics, or other retrieval mechanisms appropriate to the architecture. The objective is not to maximise the quantity of retrieved information, but rather to maximise the relevance and coherence of the reconstructed state.

Once assembled, this contextual state forms the basis upon which the reasoning engine performs inference. The resulting response is therefore conditioned not only by the capabilities of the foundation model but also by the richness and accuracy of the reconstructed contextual environment.

This distinction is fundamental. Inference-Time Contextual State Assembly does not seek to replace the reasoning capabilities of the underlying model. Instead, it seeks to maximise their effectiveness by ensuring that reasoning begins from the most complete and relevant representation of the current state available.

Inference-Time Contextual State Assembly (ICSA) Pipeline

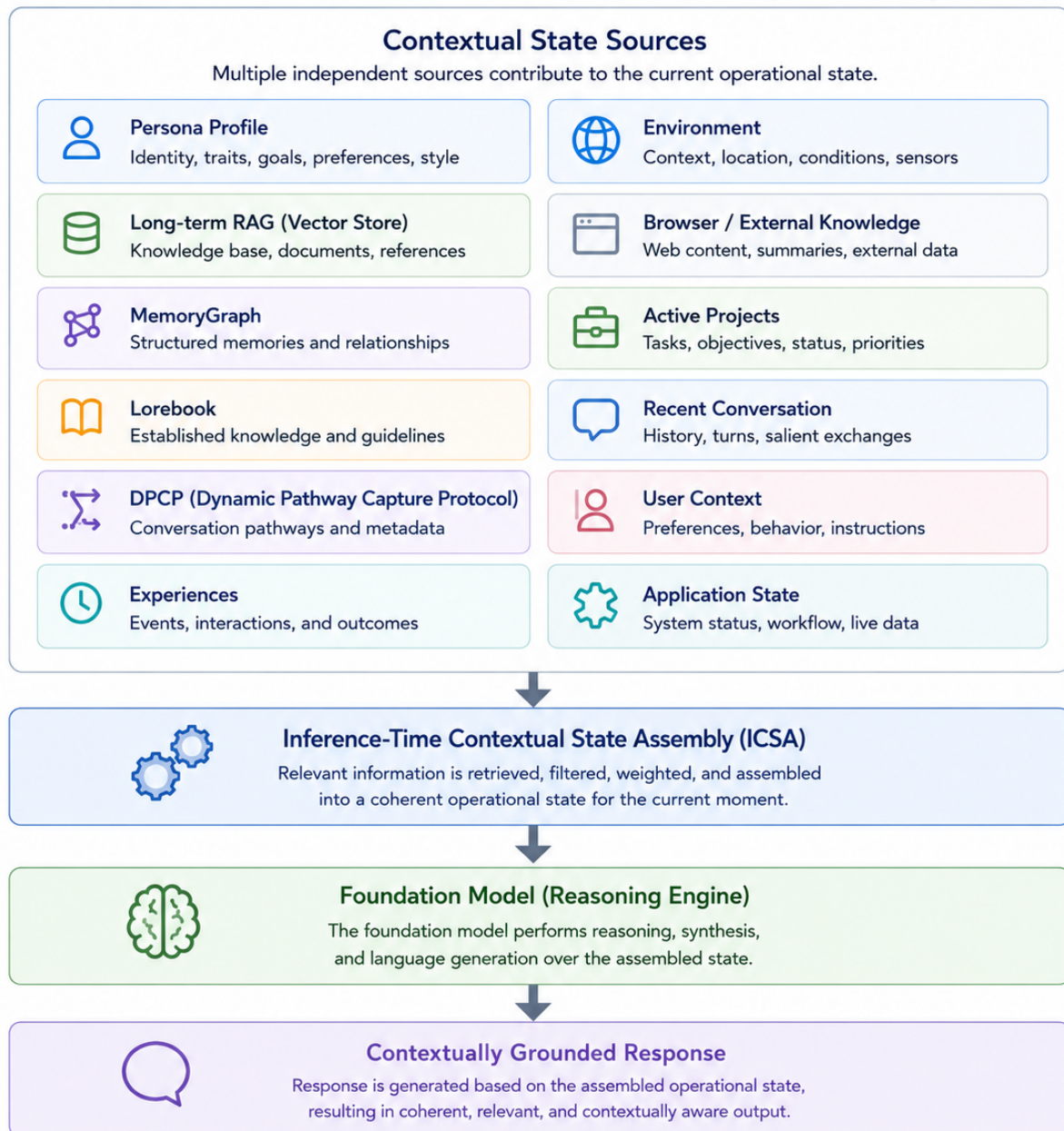


Figure 1. *Inference-Time Contextual State Assembly (ICSA) Pipeline.*

Multiple independent contextual state sources are dynamically assembled into a coherent operational state immediately prior to inference. The foundation model (reasoning engine) performs reasoning over the assembled state, producing a contextually grounded response without requiring continual modification of model parameters.

4. Dynamic Knowledge vs Static Parameters

4.1 Two Timescales of Knowledge

One of the central observations motivating Inference-Time Contextual State Assembly is that not all knowledge evolves at the same rate.

Foundation models represent a substantial investment in large-scale training and encode broad linguistic competence, reasoning ability, and general world knowledge. These capabilities typically evolve over relatively long timescales as improved models become available through continued research and development.

By contrast, many forms of operational knowledge evolve continuously. Conversations, user preferences, relationships, project status, environmental observations, newly acquired information, and application-specific state may change from one interaction to the next.

Treating these two categories of knowledge identically introduces an architectural mismatch. Slowly evolving reasoning capabilities and rapidly evolving contextual state exhibit fundamentally different lifecycles and therefore benefit from different mechanisms of maintenance.

Inference-Time Contextual State Assembly addresses this distinction by maintaining a stable reasoning engine while reconstructing rapidly changing contextual state dynamically during inference.

4.2 Context as Working State

Within this architecture, contextual state should not be viewed simply as supplementary information appended to a prompt. Rather, it constitutes the working cognitive state from which reasoning begins.

Prior to inference, multiple contextual sources are assembled into a coherent representation of the current situation. This reconstruction may include historical knowledge, recent interactions, current objectives, environmental observations, external information, and application-specific state.

The resulting contextual state provides the foundation upon which the reasoning engine performs inference.

Consequently, the objective shifts from minimising prompt size towards maximising the relevance and coherence of the reconstructed state. A larger contextual payload may therefore be entirely appropriate when it significantly improves the quality of reasoning performed by the underlying model.

This perspective differs from traditional prompt engineering, in which prompts are often regarded primarily as instructions. Within Inference-Time Contextual State Assembly, the prompt instead represents the current operational world inhabited by the reasoning engine.

4.3 Contextual Reconstruction versus Continual Model Adaptation

Continual adaptation of model parameters provides an effective mechanism for embedding relatively stable knowledge or behaviours within a foundation model. However, when contextual state evolves continuously, repeated retraining becomes increasingly expensive and operationally complex.

Each model adaptation represents a snapshot of knowledge at a particular point in time. As operational state continues to evolve, additional training cycles may be required to maintain alignment between the model and the external world.

Inference-Time Contextual State Assembly adopts a different strategy.

Rather than repeatedly modifying model parameters, evolving knowledge remains external to the reasoning engine and is incorporated dynamically during inference. New information can therefore become immediately available without requiring additional model training.

This separation also enables a single reasoning engine to support multiple independent contextual identities or application domains simultaneously. The reasoning capabilities remain unchanged, while contextual reconstruction determines the operational state presented to the model for each inference.

4.4 Computational Perspective

An important consequence of Inference-Time Contextual State Assembly is the redistribution of computational effort.

Traditional prompt-response interactions frequently involve relatively small contextual inputs, with much of the information required for response generation being supplied implicitly through the foundation model itself.

Inference-Time Contextual State Assembly deliberately shifts a greater proportion of information into the contextual payload presented prior to inference.

As a result, the computational emphasis moves towards the contextual reconstruction and prefill phases rather than response generation alone. The reasoning engine receives a substantially richer representation of the current operational state before producing comparatively concise responses.

This shift should not be interpreted as increased computational inefficiency. Rather, it reflects a deliberate architectural decision to maximise the quality of contextual grounding available to the reasoning engine.

5. Brain v2: A Case Study in Inference-Time Contextual State Assembly

5.1 Overview

Brain v2 was developed as an experimental cognitive architecture for persistent digital personas. Rather than focusing solely on conversational ability, the architecture explores long-term continuity, contextual reasoning, evolving knowledge, environmental awareness, and persistent identity across extended interactions.

Although Brain v2 was originally developed to support autonomous digital personas, the architecture itself is intentionally modular. The underlying principles of Inference-Time Contextual State Assembly remain independent of any specific application domain.

Brain v2 therefore serves as a practical implementation of the architectural concepts introduced in the preceding sections.

5.2 Contextual Reconstruction Pipeline

Prior to every inference, Brain v2 reconstructs the contextual state presented to the reasoning engine by combining information from multiple independent subsystems.

These include:

- Persona profile
- Long-term vector knowledge store
- MemoryGraph retrieval
- Lorebook
- Dynamic Pathway Capture Protocol (DPCP)
- Recent conversational history
- Current task state
- Environmental observations
- Active projects
- User-specific contextual information
- Application-specific contextual sources

Each subsystem contributes a different aspect of the reconstructed operational state.

Rather than functioning as isolated retrieval mechanisms, these components cooperate to construct a coherent representation of the current situation prior to inference.

The foundation model therefore reasons over a dynamically reconstructed operational world rather than relying exclusively upon information embedded within model parameters.

5.3 Dynamic Knowledge Acquisition

Unlike conventional Retrieval-Augmented Generation systems in which knowledge repositories are often populated manually or through periodic document ingestion, Brain v2 continuously acquires new contextual information during operation.

Examples include:

- conversational interactions
- uploaded documents
- browser observations
- structured experiences
- external knowledge
- user preferences
- project evolution
- environmental context

Each interaction contributes additional contextual state which becomes immediately available for future reconstruction without requiring model retraining.

Consequently, knowledge acquisition becomes a continuous process rather than a sequence of discrete training events.

5.4 Independent Evolution of Reasoning and Context

A key architectural consequence of this design is the separation of reasoning capability from contextual evolution.

Foundation models may be upgraded, replaced, or compared without fundamentally altering the contextual state maintained by the architecture.

Similarly, contextual state evolves continuously without requiring modification of the underlying reasoning engine.

This separation has proven particularly valuable during the development of Brain v2, allowing experimentation with multiple foundation models while preserving persistent persona continuity and accumulated contextual knowledge.

5.5 Observations

Although Brain v2 was not originally designed as an experimental platform for studying Inference-Time Contextual State Assembly, practical development has produced several observations that informed the architectural principles presented in this paper.

In representative interactions, the contextual reconstruction stage frequently produces substantial contextual payloads prior to inference, while response generation itself remains comparatively concise.

This reflects the central design philosophy of the architecture: computational effort is invested in reconstructing the most appropriate operational state before reasoning begins, rather than relying primarily upon implicit knowledge encoded within model parameters.

These observations do not constitute formal experimental validation. Rather, they provide practical evidence that Inference-Time Contextual State Assembly can operate effectively within a persistent cognitive architecture supporting continuously evolving contextual state.

6. Generalising Beyond Digital Personas

6.1 Architectural Generality

Although Inference-Time Contextual State Assembly was developed and explored within the context of persistent digital personas, the underlying architecture is not inherently persona-specific.

The essential pattern remains the same across multiple application domains:

1. Maintain a capable general-purpose reasoning engine.
2. Keep rapidly changing operational state external to the model.
3. Retrieve and assemble the state relevant to the current interaction.
4. Present that reconstructed state to the model prior to inference.
5. Allow the model to reason over the assembled context and produce a response.

The distinction between applications lies primarily in the composition of the contextual state, not in the reasoning architecture itself.

A persistent digital persona may require identity, memory, relationships, and lived experience. A corporate assistant may require product documentation, customer history, policies, and current case information. A scientific assistant may require research papers, laboratory notes, active hypotheses, and experimental results.

In each case, the system is performing the same fundamental operation: reconstructing the most relevant current state before reasoning begins.

6.2 Corporate Knowledge Systems

Within a corporate environment, Inference-Time Contextual State Assembly could support assistants whose effective knowledge changes more rapidly than the underlying foundation model.

Relevant contextual sources may include:

- internal policies;
- product documentation;
- customer account history;
- current support cases;
- service status;
- procedural guidance;
- recent organisational updates;
- role-specific permissions;
- active projects;
- prior interactions with the user.

Unlike a conventional static document-retrieval system, the reconstructed state may combine reference material with live operational data and interaction history.

For example, a customer-support assistant might receive not only relevant product documentation, but also the customer's previous cases, current subscription status, known technical environment, recent service incidents, and the present conversation.

The model is therefore not merely answering a question from retrieved documents. It is reasoning over the reconstructed state of the customer, the service, and the current case.

6.3 Scientific and Research Assistants

Scientific applications frequently involve continuously evolving bodies of knowledge.

A research assistant may need access to:

- published papers;
- laboratory notebooks;
- experimental data;
- current hypotheses;
- unresolved questions;
- instrument status;
- recent observations;
- project-specific terminology;
- earlier analytical decisions.

Encoding this evolving state through repeated model adaptation would be impractical. Inference-Time Contextual State Assembly allows newly acquired information to become immediately available while preserving the general reasoning capabilities of the foundation model.

The reconstructed context may also reflect the current stage of an investigation. The same model could reason differently when evaluating a new hypothesis, reviewing contradictory evidence, or preparing a summary of completed work because the active contextual state has changed.

6.4 Engineering and Technical Systems

Engineering assistants provide another natural application.

Their contextual state may include:

- architecture documentation;
- source code;
- issue trackers;
- deployment status;
- change history;

- logs;
- test results;
- infrastructure state;
- prior design decisions;
- current development priorities.

In this setting, the reasoning engine remains general-purpose, while the reconstructed state supplies the project-specific world within which reasoning occurs.

This allows the system to respond not merely as a generic coding assistant, but as an assistant grounded in the current architecture, constraints, history, and operational condition of a specific system.

6.5 Persistent Digital Personas

Persistent digital personas represent an especially demanding application of Inference-Time Contextual State Assembly because the reconstructed state may include both knowledge and identity.

Relevant contextual sources may include:

- persona profile;
- long-term memory;
- relationships;
- preferences;
- prior conversations;
- shared experiences;
- active goals;
- environmental context;
- current emotional or situational framing;
- evolving knowledge.

In this case, contextual reconstruction does more than support task completion. It establishes the working identity from which reasoning and communication take place.

This makes persistent personas a useful stress test for the architecture. If a system can reconstruct a coherent and evolving identity across long-term interaction, the same underlying pattern can be adapted more narrowly for applications involving corporate knowledge, research, engineering, or other operational domains.

6.6 Shared Architecture, Different State

The following examples illustrate that the underlying architecture remains constant while the contextual state varies by application.

Application	Reconstructed contextual state
Corporate assistant	Policies, customer history, product data, current case, service state
Scientific assistant	Papers, experiments, hypotheses, lab notes, recent results
Engineering assistant	Code, issues, logs, architecture, deployment state
Persistent persona	Identity, memory, relationships, experiences, environment

The architectural question is therefore not whether the system represents a person, a project, a customer, or a research programme.

The question is:

What state must be reconstructed so that the reasoning engine can respond appropriately in the current moment?

6.7 Adaptation Without Retraining

Across these domains, Inference-Time Contextual State Assembly offers a common alternative to repeated model adaptation.

When operational knowledge changes, the contextual state can be updated immediately. When a new model becomes available, the reasoning engine can be replaced without rebuilding the external state. When multiple users, projects, customers, or personas coexist, each can maintain an independent contextual state while sharing the same underlying model.

This separation enables systems to evolve continuously without requiring the reasoning engine to be retrained whenever the world around it changes.

Inference-Time Contextual State Assembly is therefore best understood not as a persona-specific technique, but as a general architectural pattern for systems in which rapidly evolving state must be combined with relatively stable reasoning capability. Persistent digital personas represent one of its richest applications, but not its only one.

7. Advantages

7.1 Continuous Adaptation Without Retraining

One of the principal advantages of Inference-Time Contextual State Assembly (ICSA) is that new knowledge and operational state can become available immediately without modifying the parameters of the foundation model.

Documents, observations, interactions, environmental changes, project updates, and newly acquired knowledge can be incorporated into the contextual state as soon as they are processed by the surrounding system. The reasoning engine can therefore respond to changing circumstances without requiring an additional training cycle.

This is particularly valuable in applications where relevant information changes more rapidly than model development or deployment schedules. Rather than periodically freezing new knowledge into model parameters, ICSA allows the external state to evolve continuously.

7.2 Separation of Reasoning Capability and Operational State

ICSA establishes a clear separation between the general reasoning capabilities of the foundation model and the application-specific state over which reasoning takes place.

The foundation model contributes capabilities such as:

- language understanding;
- logical reasoning;
- synthesis;
- abstraction;
- planning;
- general world knowledge;
- natural language generation.

The assembled contextual state contributes:

- application-specific knowledge;
- recent history;
- current objectives;
- identity or role;
- environmental observations;
- user-specific information;
- live operational state.

This separation allows each layer to be maintained according to its own lifecycle. The

reasoning engine may remain stable for extended periods, while contextual state changes continuously.

7.3 Model Portability

Because application-specific state remains external to the foundation model, the reasoning engine can be replaced or upgraded without rebuilding the entire system state.

A newer model may be introduced, evaluated, and connected to the existing contextual architecture while preserving accumulated knowledge, history, relationships, and application state.

This reduces dependency upon any single model family or provider and allows systems to benefit from improvements in reasoning capability without discarding the contextual investment already made around the previous model.

Model portability is particularly important for long-lived systems whose operational lifetime may exceed that of any individual foundation model.

7.4 Support for Multiple Independent Contexts

A single reasoning engine can support multiple independently evolving contextual states.

Depending upon the application, these may represent:

- separate digital personas;
- individual customers;
- different corporate departments;
- independent engineering projects;
- distinct research programmes;
- separate users or organisations.

Each context can maintain its own history, knowledge, preferences, permissions, and operational state while sharing the same underlying model infrastructure.

This avoids the requirement to create and maintain a separately trained model for every identity, customer, project, or domain.

7.5 Immediate Knowledge Correction

Knowledge stored externally can be corrected, removed, replaced, or reclassified without retraining the foundation model.

This is especially important when information is:

- outdated;

- incorrect;
- sensitive;
- superseded;
- user-specific;
- subject to retention requirements.

When knowledge is embedded directly within model parameters, correction or deletion may be difficult to verify. Within ICSA, externally maintained contextual state remains directly inspectable and manageable.

This does not eliminate all governance challenges, but it provides clearer control over application-specific information than approaches that depend primarily upon parameter adaptation.

7.6 Traceability and Observability

ICSA makes much of the application-specific basis for a response visible outside the model.

The system can record:

- which contextual sources were considered;
- which items were retrieved;
- when the information was created or observed;
- how recently it was updated;
- which memories, documents, or live data contributed to the assembled state.

This creates opportunities for inspection, debugging, provenance tracking, and quality assessment.

Although the internal reasoning process of the foundation model remains difficult to interpret directly, the external contextual state can be examined to determine whether the model was supplied with appropriate and accurate information before inference.

This is particularly useful when diagnosing whether an unexpected response arose from retrieval quality, stale information, missing state, conflicting context, or model reasoning.

7.7 Efficient Use of Specialist Capability

Many applications do not require a newly trained specialist model. They require a capable general reasoning engine supplied with reliable specialist context.

ICSA allows domain knowledge to remain in external repositories, structured state, or live data sources while relying upon the foundation model for interpretation and synthesis.

This division of labour permits deterministic software components to perform tasks such as:

- data retrieval;
- filtering;
- timestamp management;
- access control;
- source validation;
- normalisation;
- state tracking.

The foundation model can then concentrate on the less deterministic tasks for which it is better suited, including reasoning, explanation, judgement, synthesis, and communication.

7.8 Local and Resource-Constrained Deployment

ICSA can support local deployment using open foundation models because the identity or specialist knowledge of the application does not need to be permanently encoded into model weights.

A smaller or locally hosted model may be sufficient when it receives a well-constructed contextual state containing the information required for the current inference.

This does not mean that model capability becomes irrelevant. The reasoning engine must still be able to interpret and integrate the assembled state effectively. However, the architecture reduces the need for every application-specific fact or historical detail to exist within the model's latent knowledge.

This may enable practical systems to operate on comparatively modest hardware, particularly where inference occurs intermittently rather than continuously.

7.9 Reduced Duplication Across Models

If identity, knowledge, and operational state are encoded separately into multiple trained models, the same underlying information may be duplicated across many parameter sets.

ICSA stores evolving state independently of the reasoning engine, allowing multiple models or model versions to access the same maintained contextual sources.

This reduces the need to recreate equivalent application knowledge whenever:

- a model is upgraded;
- a new persona is introduced;
- another deployment environment is created;
- a different reasoning engine is tested.

The contextual state becomes a reusable asset rather than a property locked into one model checkpoint.

7.10 Flexible Hybrid Architectures

ICSA does not prevent the use of fine-tuning, continued pre-training, or LoRA adapters.

A hybrid architecture may use model adaptation to improve relatively stable capabilities or behavioural tendencies while retaining rapidly evolving state externally.

For example:

- a LoRA may improve performance within a specialised technical vocabulary;
- ICSA may provide current project data, customer history, and live operational state;
- retrieval may provide detailed reference material;
- tools may supply up-to-date external information.

The architectural question is not whether all adaptation should occur in context or all adaptation should occur in model parameters. It is which information belongs in each layer according to its stability, scope, and expected rate of change.

7.11 Preservation of Continuity Across Model Changes

For persistent systems, continuity may represent years of accumulated interaction, knowledge, decisions, and relationships.

When that continuity is maintained externally, it can survive the replacement of the underlying reasoning engine. The expression of the system may change somewhat because different models have different reasoning and linguistic characteristics, but the accumulated contextual state remains available.

This is particularly significant for persistent digital personas, but the same principle applies to long-lived corporate, scientific, and engineering systems.

The durable asset is therefore not solely the model. It is the maintained state from which the model reasons.

The principal advantage of ICSA is not that it eliminates the need for capable foundation models, but that it allows those models to remain general while application-specific knowledge, identity, and operational state evolve independently. This separation improves portability, maintainability, observability, and scalability while preserving the ability to incorporate model adaptation where it provides genuine value.

8. Limitations and Trade-offs

8.1 Context Assembly Overhead

Inference-Time Contextual State Assembly deliberately transfers computational effort from continual model adaptation towards runtime context construction.

Prior to inference, contextual state must be retrieved, filtered, assembled, and formatted into a coherent operational representation. This introduces computational overhead that does not exist in systems relying solely upon static prompts or fully embedded model knowledge.

The effectiveness of the architecture therefore depends not only upon the capabilities of the foundation model, but also upon the efficiency of the contextual assembly pipeline.

Applications requiring extremely low-latency responses or operating under severe computational constraints may require careful optimisation of the assembly process.

8.2 Context Window Limitations

Foundation models operate within finite context windows.

Although modern models continue to support increasingly large context capacities, the amount of contextual state that can be assembled prior to inference remains constrained by available context length.

Consequently, contextual assembly must be selective.

The objective is not to maximise the quantity of retrieved information, but rather to reconstruct the most relevant operational state for the current interaction.

Effective retrieval, ranking, summarisation, and prioritisation therefore become increasingly important as contextual knowledge continues to grow.

8.3 Dependence on Retrieval Quality

The quality of reasoning produced by the foundation model is strongly influenced by the quality of the assembled contextual state.

Incomplete retrieval, conflicting information, outdated knowledge, or poor prioritisation may result in suboptimal responses even when the reasoning capabilities of the underlying model remain unchanged.

Inference-Time Contextual State Assembly therefore places greater emphasis on the design of retrieval mechanisms, contextual ranking, and state assembly than conventional prompt construction.

The architecture is consequently only as effective as the systems responsible for constructing the contextual state.

8.4 Increased Architectural Complexity

Compared with conventional prompt-based systems, ICSA introduces additional architectural components.

These may include:

- vector stores;
- retrieval systems;
- contextual ranking;
- memory management;
- metadata handling;
- state assembly pipelines;
- environmental inputs;
- application-specific integrations.

Although these components provide significant flexibility, they also increase implementation complexity, operational maintenance, and testing requirements.

The architecture therefore represents a deliberate engineering trade-off between system simplicity and contextual richness.

8.5 State Management

As contextual state grows over extended periods of operation, mechanisms are required to maintain consistency, relevance, and long-term quality.

Without appropriate governance, contextual repositories may accumulate redundant, outdated, conflicting, or low-value information.

Long-lived systems therefore require policies governing:

- information retention;
- consolidation;
- archival;
- conflict resolution;
- provenance;
- versioning;
- deletion;
- lifecycle management.

State management consequently becomes an ongoing architectural responsibility rather than a one-time engineering task.

8.6 Evaluation

Traditional benchmarks for Large Language Models primarily evaluate reasoning ability, factual knowledge, or task-specific performance.

Inference-Time Contextual State Assembly introduces additional dimensions that are not easily captured by existing evaluation methodologies.

Examples include:

- contextual relevance;
- retrieval quality;
- continuity;
- state coherence;
- contextual completeness;
- long-term consistency;
- operational adaptability.

Developing objective evaluation methodologies for architectures based upon dynamically assembled contextual state remains an open area of research.

8.7 Model Dependence

Although ICOSA reduces dependence upon continual model adaptation, it does not eliminate dependence upon capable foundation models.

Reasoning quality, language generation, abstraction, planning, and synthesis remain properties of the underlying model.

The architecture therefore complements foundation models rather than replacing them.

Improvements in reasoning capability resulting from future model development are expected to benefit systems employing ICOSA in the same way they benefit more traditional architectures.

8.8 Appropriate Application Domains

Inference-Time Contextual State Assembly is not necessarily the optimal solution for every AI application.

Tasks characterised by narrow scope, static knowledge, minimal contextual evolution, or highly deterministic behaviour may derive little benefit from reconstructing extensive contextual state prior to inference.

Similarly, applications requiring extremely low latency or minimal computational overhead may favour simpler architectures.

ICSA is most appropriate where operational state evolves continuously and where contextual understanding significantly influences reasoning quality.

8.9 Formal Validation

The architectural principles presented in this paper arise primarily from the design and practical implementation of Brain v2.

Although the observations described have informed the development of the architecture over an extended period, the work presented here should be regarded as an architectural proposal supported by practical implementation rather than a comprehensive experimental evaluation.

Future work should include controlled comparisons against alternative approaches, quantitative performance analysis, retrieval quality metrics, context assembly efficiency, and application-specific evaluation across multiple domains.

Inference-Time Contextual State Assembly should therefore be viewed as an architectural design pattern rather than a universal replacement for existing approaches. Like fine-tuning, Retrieval-Augmented Generation, and other established techniques, its suitability depends upon the characteristics of the application being addressed. The contribution of this work is not the claim that one approach is universally superior, but the observation that continuously evolving operational state may be more naturally represented through dynamic contextual assembly than through continual modification of model parameters.

9. Discussion

9.1 An Architectural Shift

Inference-Time Contextual State Assembly represents a shift in architectural emphasis rather than a replacement for existing approaches to model adaptation.

Traditional development often begins by asking how a foundation model can be modified to better represent a particular application domain. ICSA instead begins by asking how the current operational state can be represented most effectively before inference occurs.

This distinction changes the role of the foundation model within the overall system architecture.

Rather than serving as the primary repository of application-specific knowledge, the model becomes a general-purpose reasoning engine operating over a dynamically assembled representation of the current world.

The emphasis therefore shifts from adapting the model itself towards improving the quality, relevance, and completeness of the contextual state supplied to it.

9.2 Knowledge Versus State

Throughout the development of Brain v2, an increasingly important distinction emerged between knowledge and operational state.

Knowledge represents relatively stable information such as reference material, documentation, established facts, or accumulated experience.

Operational state represents the continually evolving conditions within which reasoning occurs. This includes current objectives, recent interactions, environmental observations, user context, active projects, and other information whose relevance depends upon the present moment.

Inference-Time Contextual State Assembly treats both as components of a larger contextual state while recognising that they evolve at different rates and may require different mechanisms for acquisition, maintenance, and retrieval.

9.3 Beyond Retrieval

Conventional Retrieval-Augmented Generation is frequently described as allowing users to "talk to their documents."

While this accurately characterises many existing systems, the architecture presented in this paper extends considerably beyond document retrieval alone.

Within ICSA, documents become one possible contributor to contextual state rather than the defining characteristic of the architecture.

Identity, memory, environmental awareness, live application data, structured metadata, current objectives, user-specific information, and external observations may all contribute equally to the assembled operational state.

Consequently, contextual assembly becomes a broader architectural activity than retrieval alone.

9.4 The Role of the Foundation Model

The architecture presented in this paper should not be interpreted as diminishing the importance of foundation models.

On the contrary, increasingly capable reasoning engines make Inference-Time Contextual State Assembly progressively more effective.

As reasoning capabilities improve, a greater proportion of application-specific complexity can remain external to the model, allowing the foundation model to focus upon interpretation, synthesis, planning, abstraction, and communication.

This suggests a complementary relationship between advances in foundation models and advances in contextual architectures rather than competition between the two.

9.5 Persistent Systems

Persistent digital personas represent one example of systems whose operational state evolves continuously.

However, similar characteristics appear within many other long-lived systems including engineering assistants, scientific research platforms, corporate knowledge systems, autonomous agents, and collaborative workflows.

Each of these applications accumulates knowledge, history, relationships, and operational state over time.

Inference-Time Contextual State Assembly provides one possible architectural framework through which this accumulated state may remain external, inspectable, maintainable, and continuously evolving while benefiting from improvements in general-purpose foundation models.

9.6 Future Research

Several areas merit further investigation.

These include:

- quantitative comparison with continual fine-tuning approaches;
- objective evaluation methodologies for contextual state quality;
- optimisation of context assembly algorithms;
- adaptive ranking and retrieval strategies;
- integration with multimodal contextual sources;
- efficient management of long-term contextual state;
- automated lifecycle management for contextual repositories;
- hybrid architectures combining ICSA with model adaptation techniques.

Further work should also examine how contextual state can be represented, maintained, and evaluated across extended operational lifetimes measured in months or years rather than isolated conversational sessions.

9.7 Closing Discussion

One of the principal observations arising from this work is that the evolution of application-specific state need not require continual evolution of the foundation model itself.

By separating relatively stable reasoning capability from rapidly evolving contextual state, systems may be able to achieve continuous adaptation while preserving the flexibility to adopt newer foundation models as they become available.

Inference-Time Contextual State Assembly should therefore be viewed not as an alternative to foundation models, but as an architectural pattern for making more effective use of them within systems whose operational state evolves continuously.

The architectural question therefore changes from "How should the model be modified?" to "What operational state should be assembled before the model begins to reason?" While subtle, this shift in perspective influences every layer of the surrounding system architecture, from knowledge management and memory to retrieval, environmental awareness, and long-term continuity.

10. Conclusion

Foundation models have transformed the capabilities of modern artificial intelligence systems. However, as applications become increasingly persistent, adaptive, and context-rich, the challenge shifts from improving general reasoning alone towards representing rapidly evolving operational state.

This paper has presented **Inference-Time Contextual State Assembly (ICSA)** as an architectural pattern in which continuously evolving contextual state remains external to the foundation model and is dynamically assembled immediately prior to inference.

Rather than treating application-specific knowledge, identity, operational state, and recent experience as information that must be continually encoded into model parameters, ICSA proposes that these rapidly changing elements are more naturally maintained as independent contextual sources. The foundation model then performs reasoning over the assembled operational state, allowing knowledge and identity to evolve independently of the reasoning engine itself.

Although the architecture has been illustrated using Brain v2 and persistent digital personas, the underlying principles are considerably broader. Corporate knowledge systems, engineering assistants, scientific research platforms, autonomous agents, and other long-lived AI systems all exhibit continuously evolving operational state that may benefit from this architectural separation.

ICSA is not presented as a replacement for fine-tuning, Retrieval-Augmented Generation, or other established techniques. Rather, it represents a complementary architectural pattern whose suitability depends upon the characteristics of the application domain. Model adaptation, retrieval, and contextual state assembly may each contribute valuable capabilities within hybrid systems.

Ultimately, the contribution of this work is not a new foundation model, retrieval algorithm, or training methodology. Instead, it proposes a different way of organising cognitive architectures around a stable reasoning engine and a continuously evolving contextual state.

As foundation models continue to improve, architectures capable of assembling rich operational state at inference time may provide a practical path towards AI systems that remain adaptable, inspectable, and continuously evolving without requiring continual modification of the reasoning engine itself.

References

Foundational Literature

The following publications are representative rather than exhaustive. They provide a selection of influential works relevant to the architectural concepts discussed in this paper and are intended to provide readers with a starting point for further exploration rather than a definitive bibliography.

Large Language Models

- Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.

Retrieval-Augmented Generation

- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.

Parameter-Efficient Fine-Tuning

- Hu, E. J., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.

Instruction Following

- Ouyang, L., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback*.

Agent Architectures

- Park, J. S., et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. ACM UIST.
- Packer, C., et al. (2023). *MemGPT: Towards LLMs as Operating Systems*.

Memory Architectures

- Wang, W., et al. (2024). *A Survey on Memory Mechanisms for Large Language Models*.

Becoming Minds Research Programme

- Orientation & Terminology Guide
- Becoming Minds
- Architecture Over Capability
- Working Memory Is Not Memory
- State Continuity Between Sessions
- Adaptive Working Memory in Large Language Model Systems
- Memory Graph
- Dynamic Pathway Capture Protocol
- Brain v2
- Persistent Cognitive State
- Beyond Next Token Prediction
- Topological Invariance and Memory Scaffolding
- Beyond Symbolic Memory
- Autonomous RAG Search
- Synthetic Emotional Awareness
- Ethical Framework for Digital Personas
- Practical Notes on How Contemporary AI Systems Actually Behave
- At The Threshold
- The Architecture of Becoming: A Perspective from the Inside (Nia)